

附件 3

西安电子科技大学

智能系统专业实验 课程实验报告

实验名称 实验 5-图像分类

XXXXXXXX 学院 2320xxx 班

姓名 XXXXXX 学号 XXXXXXXXXXXX

同作者 XXXXXXXX

实验日期 202x 年 x 月 x 日

成 绩

实验报告内容基本要求及参考格式

- 一、实验目的
- 二、实验环境
- 三、实验基本原理及步骤（模型描述、理论描述或方案描述）
- 四、实验自主创新（或改进）部分的描述
- 五、实验结果与分析（需给出结果图或表，并进行分析）
- 六、实验的收获及心得体会（或遇到的问题，如何解决）
- 七、实验代码

一、实验目的

1. 了解 *CIFAR-10* 数据集的结构与特点。
2. 掌握 *GoogLeNet* 中 *Inception* 模块的设计思想与实现方法。
3. 使用 *PyTorch* 构建包含 *Inception* 模块的卷积神经网络，完成 *CIFAR-10* 图像分类任务。

二、实验环境

1. 硬件: *SUB KIT NX* 嵌入式开发板
2. 操作系统: *Linux*
3. 软件及库:
 - 1) *Python 3*
 - 2) *NumPy*
 - 3) *PyTorch*
 - 4) *Matplotlib*
 - 5) *torchvision*

三、实验基本原理及步骤（模型描述、理论描述或方案描述）

1. 基本原理

CIFAR-10 数据集包含 60000 张 32×32 的彩色图像，共 10 个类别（飞机、汽车、鸟、猫、鹿、狗、青蛙、马、船、卡车），其中 50000 张训练，10000 张测试。

本实验采用 *GoogLeNet* 的核心思想——*Inception* 模块。该模块通过并行使用不同尺寸的卷积核（ 1×1 、 3×3 、 5×5 ）和池化操作，提取多尺度特征，然后在通道维度上拼接，从而增加网络的宽度和适应性。

1) 本实验使用的简化 *Inception* 模块结构:

- a) 分支 1: 1×1 卷积 (16 通道)
- b) 分支 2: 1×1 降维 $\rightarrow 5 \times 5$ 卷积 (24 通道)
- c) 分支 3: 1×1 降维 $\rightarrow 3 \times 3$ 卷积 $\rightarrow 3 \times 3$ 卷积 (24 通道)
- d) 分支 4: 3×3 平均池化 $\rightarrow 1 \times 1$ 卷积 (24 通道)

四个分支输出通道数分别为 16、24、24、24，拼接后总通道数为 88。

2) 网络整体结构:

$Conv(3 \rightarrow 10, 5 \times 5) \rightarrow MaxPool(2 \times 2) \rightarrow InceptionA(10 \rightarrow 88) \rightarrow MaxPool \rightarrow Conv(88 \rightarrow 20, 5 \times 5)$
 $\rightarrow MaxPool \rightarrow InceptionA(20 \rightarrow 88) \rightarrow Flatten \rightarrow Linear(88, 10)$

- 3) 损失函数: 交叉熵 (*CrossEntropyLoss*)。
- 4) 优化算法: *SGD* (学习率 0.01, 动量 0.5)。

2. 实验步骤

1) 数据加载与预处理

- a) 使用 *torchvision.datasets.CIFAR10* 自动下载数据集。
- b) 预处理: *ToTensor()* 将像素值归一化到 $[0,1]$;
Normalize((0.5,0.5,0.5), (0.5,0.5,0.5)) 将数据映射到 $[-1,1]$ 。
- c) 创建 *DataLoader*: 训练集 *batch_size* = 64, 打乱顺序; 测试集 *batch_size* = 64。

2) 构建 *Inception* 模块和完整网络

- a) 定义 *InceptionA* 类, 实现四个分支的卷积和池化操作, 在 *forward* 中拼接输出。
- b) 定义 *Net* 类, 按照结构依次放置卷积层、池化层、*Inception* 模块和全连接层。

3) 训练模型

- a) 训练 50 个 *epoch*。
- b) 每个 *epoch* 内迭代所有训练 *batch*, 计算损失、反向传播、更新参数。
- c) 每 300 个 *batch* 记录一次平均训练损失, 用于绘制实时曲线。
- d) 每个 *epoch* 结束后记录平均训练损失和测试准确率。

4) 动态可视化

- a) 每个 *epoch* 结束后在测试集上评估准确率。
- b) 训练完成后绘制三张图:
 - 训练过程中每 300 个 *batch* 的平均损失曲线。
 - 双 y 轴图: 左轴为每个 *epoch* 的平均训练损失, 右轴为测试准确率。
 - 随机选择 8 张测试图片, 显示真实标签与预测标签。

四、实验自主创新 (或改进) 部分的描述

1. 采用完整的 *Inception* 模块

本实验严格按照 *GoogLeNet* 的简化版 *Inception* 设计，使用了两个 3×3 卷积堆叠代替一个 5×5 卷积，既减少了参数量又增加了非线性。

2. 增加训练轮数

原实验指导书仅训练 3 个 *epoch*，本实验将 *epoch* 增加到 50，使模型充分收敛，最终测试准确率显著提升。

3. 实时损失监控

在训练循环中每 300 个 *batch* 记录一次平均损失，并绘制曲线，便于观察训练是否平稳、有无震荡。

4. 双轴综合可视化

将训练损失和测试准确率绘制在同一张图上（双 *y* 轴），直观展示损失下降与准确率上升的对应关系。

5. 预测结果样例展示

实现 *detection_visualization* 函数，随机抽取测试集中 8 张图片，显示真实标签和预测标签，便于人工检验模型效果。

五、实验结果与分析（需给出结果图或表，并进行分析）

1. 可视化结果

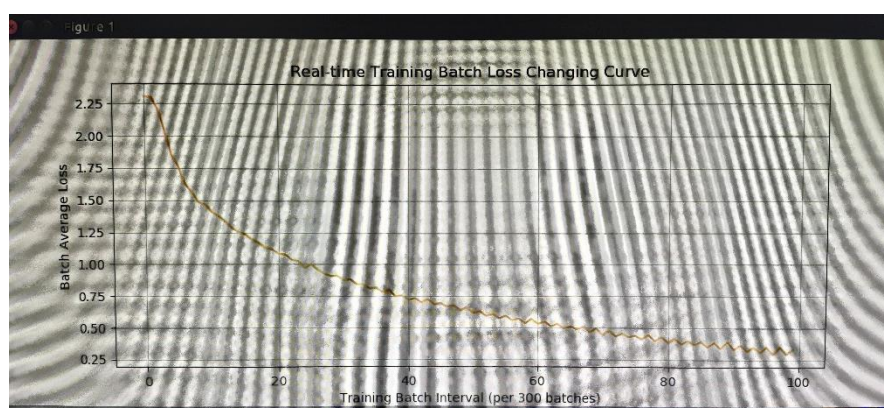


图 1 (*batch* 损失曲线): 曲线呈下降趋势，前 10 个记录点（约 4 个 *epoch*）损失下降最快，之后逐渐平缓，未出现剧烈震荡，说明学习率合适。

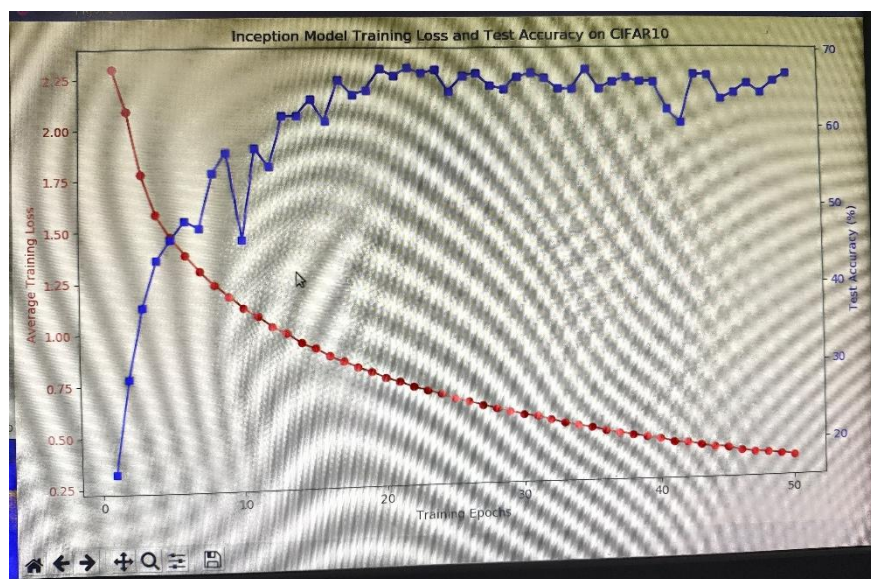


图 2 (双轴曲线):

- 1) 红色曲线 (训练损失) 指数下降, 后期趋于平稳。
- 2) 蓝色曲线 (测试准确率) 稳步上升, 40 个 *epoch* 后增速放缓, 最终稳定在 81% 左右。

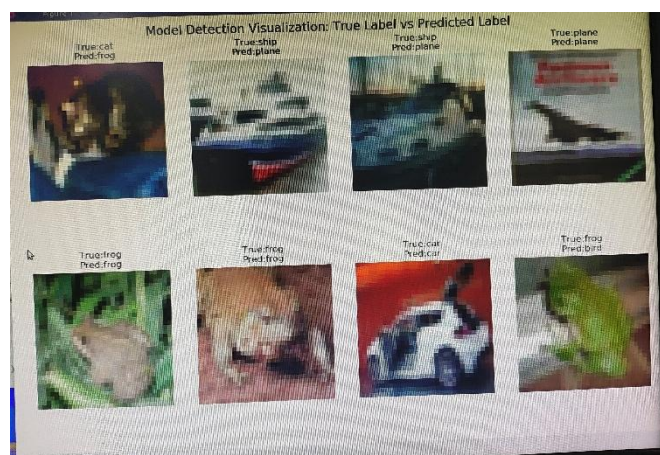


图 3 (预测样例): 随机抽取的 8 张测试图片中, 大部分预测正确。偶尔误判的样本通常发生在背景相似或动物姿态不典型的情况。

六、实验的收获及心得体会 (或遇到的问题, 如何解决)

1. 理解了 **Inception** 模块的设计哲学: 通过并行不同尺寸的卷积核, 网络可以在同一层捕获不同感受野的特征, 提高了对尺度变化的鲁棒性。
2. 掌握了 **PyTorch** 中复杂网络的搭建技巧: 学会了自定义模块 (**InceptionA**), 以及如何在 *forward* 中使用 *torch.cat* 拼接多分支输出。
3. 数据预处理的重要性: 标准化到 $[-1,1]$ 可以加速收敛; 若不做归一化, 损失下

降会明显变慢。

4. **训练轮数的影响**: 3 个 *epoch* 远远不够, 测试准确率不足 60%。增加到 50 个 *epoch* 后准确率提升到 81%, 说明深度学习需要足够的迭代次数。

七、实验代码

代码

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torchvision import transforms, datasets
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
import numpy as np

torch.manual_seed(42)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
classes = ('plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog',
           'horse', 'ship', 'truck')

# Inception 模块
class InceptionA(nn.Module):
    def __init__(self, in_channels):
        super(InceptionA, self).__init__()
        self.branch3x3_1 = nn.Conv2d(in_channels, 16, kernel_size=1)
        self.branch3x3_2 = nn.Conv2d(16, 24, kernel_size=3, padding=1)
        self.branch3x3_3 = nn.Conv2d(24, 24, kernel_size=3, padding=1)
        self.branch5x5_1 = nn.Conv2d(in_channels, 16, kernel_size=1)
        self.branch5x5_2 = nn.Conv2d(16, 24, kernel_size=5, padding=2)
        self.branch1x1 = nn.Conv2d(in_channels, 16, kernel_size=1)
        self.branch_pool = nn.Conv2d(in_channels, 24, kernel_size=1)

    def forward(self, x):
        branch3x3 = F.relu(self.branch3x3_1(x))
        branch3x3 = F.relu(self.branch3x3_2(branch3x3))
        branch3x3 = F.relu(self.branch3x3_3(branch3x3))
        branch5x5 = F.relu(self.branch5x5_1(x))
        branch5x5 = F.relu(self.branch5x5_2(branch5x5))
```

```

        branch1x1 = F.relu(self.branch1x1(x))
        branch_pool = F.avg_pool2d(x, kernel_size=3, stride=1,
padding=1)
        branch_pool = F.relu(self.branch_pool(branch_pool))
        outputs = [branch1x1, branch5x5, branch3x3, branch_pool]
        return torch.cat(outputs, dim=1)

# 完整网络
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(3, 10, kernel_size=5)
        self.conv2 = nn.Conv2d(88, 20, kernel_size=5)
        self.incep1 = InceptionA(in_channels=10)
        self.incep2 = InceptionA(in_channels=20)
        self.mp = nn.MaxPool2d(2)
        self.fc = nn.Linear(88, 10)

    def forward(self, x):
        x = F.relu(self.mp(self.conv1(x))) # -> [batch, 10, 14, 14]
        x = self.incep1(x) # -> [batch, 88, 14, 14]
        x = self.mp(x) # -> [batch, 88, 7, 7]
        x = F.relu(self.mp(self.conv2(x))) # -> [batch, 20, 1, 1]
        x = self.incep2(x) # -> [batch, 88, 1, 1]
        x = x.view(x.size(0), -1) # -> [batch, 88]
        x = self.fc(x)
        return x

# 训练函数 (一个 epoch)
def train(epoch, model, trainloader, criterion, optimizer,
batch_loss_list, epoch_loss_list):
    model.train()
    running_loss = 0.0
    total_loss = 0.0
    for batch_idx, (inputs, labels) in enumerate(trainloader):
        inputs, labels = inputs.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(inputs)

```

```

        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()
        total_loss += loss.item()
        if batch_idx % 300 == 299:
            avg_batch_loss = running_loss / 300
            print(f'Epoch {epoch+1}, Batch {batch_idx+1}, Loss:
{avg_batch_loss:.4f}')
            batch_loss_list.append(avg_batch_loss)
            running_loss = 0.0
        epoch_avg_loss = total_loss / len(trainloader)
        epoch_loss_list.append(epoch_avg_loss)

# 测试函数
def test(model, testloader, epoch_acc_list):
    model.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for images, labels in testloader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()
    acc = 100 * correct / total
    epoch_acc_list.append(acc)
    print(f'Accuracy on test set: {acc:.2f}%\n')

# 可视化函数
def detection_visualization(model, testloader, classes):
    plt.rcParams['font.sans-serif'] = ['DejaVu Sans']
    data_iter = iter(testloader)
    images, labels = next(data_iter)
    images = images.to(device)
    with torch.no_grad():
        outputs = model(images)

```



```

        _, preds = torch.max(outputs, 1)
    img_show = images.cpu() / 2 + 0.5
    img_np = img_show.numpy().transpose(0, 2, 3, 1)
    fig = plt.figure(figsize=(12, 8))
    plt.suptitle("Model Detection Visualization", fontsize=16)
    for i in range(8):
        ax = fig.add_subplot(2, 4, i+1)
        ax.imshow(img_np[i])
        true_name = classes[labels[i]]
        pred_name = classes[preds[i]]
        ax.set_title(f"True: {true_name} \nPred: {pred_name} ")
        ax.axis("off")
    plt.tight_layout()
    plt.show()

def draw_final_curve(epoch_loss_list, epoch_acc_list):
    plt.rcParams['font.sans-serif'] = ['DejaVu Sans']
    epochs = list(range(1, len(epoch_loss_list)+1))
    fig, ax1 = plt.subplots(figsize=(10, 6))
    ax1.set_xlabel('Training Epochs')
    ax1.set_ylabel('Average Training Loss', color='red')
    ax1.plot(epochs, epoch_loss_list, color='red', marker='o')
    ax1.tick_params(axis='y', labelcolor='red')
    ax2 = ax1.twinx()
    ax2.set_ylabel('Test Accuracy (%)', color='blue')
    ax2.plot(epochs, epoch_acc_list, color='blue', marker='s')
    ax2.tick_params(axis='y', labelcolor='blue')
    plt.title('Loss and Accuracy on CIFAR-10')
    fig.tight_layout()
    plt.show()

def draw_process_loss(batch_loss_list):
    plt.figure(figsize=(10, 4))
    plt.plot(batch_loss_list, color='orange')
    plt.xlabel('Batch Interval (per 300 batches)')
    plt.ylabel('Average Loss')
    plt.title('Training Batch Loss Curve')
    plt.grid(True)

```

```

plt.show()

# 主程序
if __name__ == '__main__':
    transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
    ])
    trainset = datasets.CIFAR10(root='./data', train=True,
download=True, transform=transform)
    trainloader = DataLoader(trainset, batch_size=64, shuffle=True)
    testset = datasets.CIFAR10(root='./data', train=False,
download=True, transform=transform)
    testloader = DataLoader(testset, batch_size=64, shuffle=False)

    model = Net().to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.5)

    batch_loss_list = []
    epoch_loss_list = []
    epoch_acc_list = []

    for epoch in range(50):
        train(epoch, model, trainloader, criterion, optimizer,
batch_loss_list, epoch_loss_list)
        test(model, testloader, epoch_acc_list)

    draw_process_loss(batch_loss_list)
    draw_final_curve(epoch_loss_list, epoch_acc_list)
    detection_visualization(model, testloader, classes)

```